

MDC-102 Android：Material Design 結構和版面配置 (Java)

來源：<https://codelabs.developers.google.com/codelabs/mdc-102-java?hl=zh-tw>

步驟數：6

瞭解如何在 Java 上使用 Android 的 Material 結構和版面配置。

目錄

1. [簡介](#)
2. [設定開發環境](#)
3. [新增頂端應用程式列](#)
4. [新增卡片](#)
5. [建立卡片格線](#)
6. [重點回顧](#)

1. 簡介



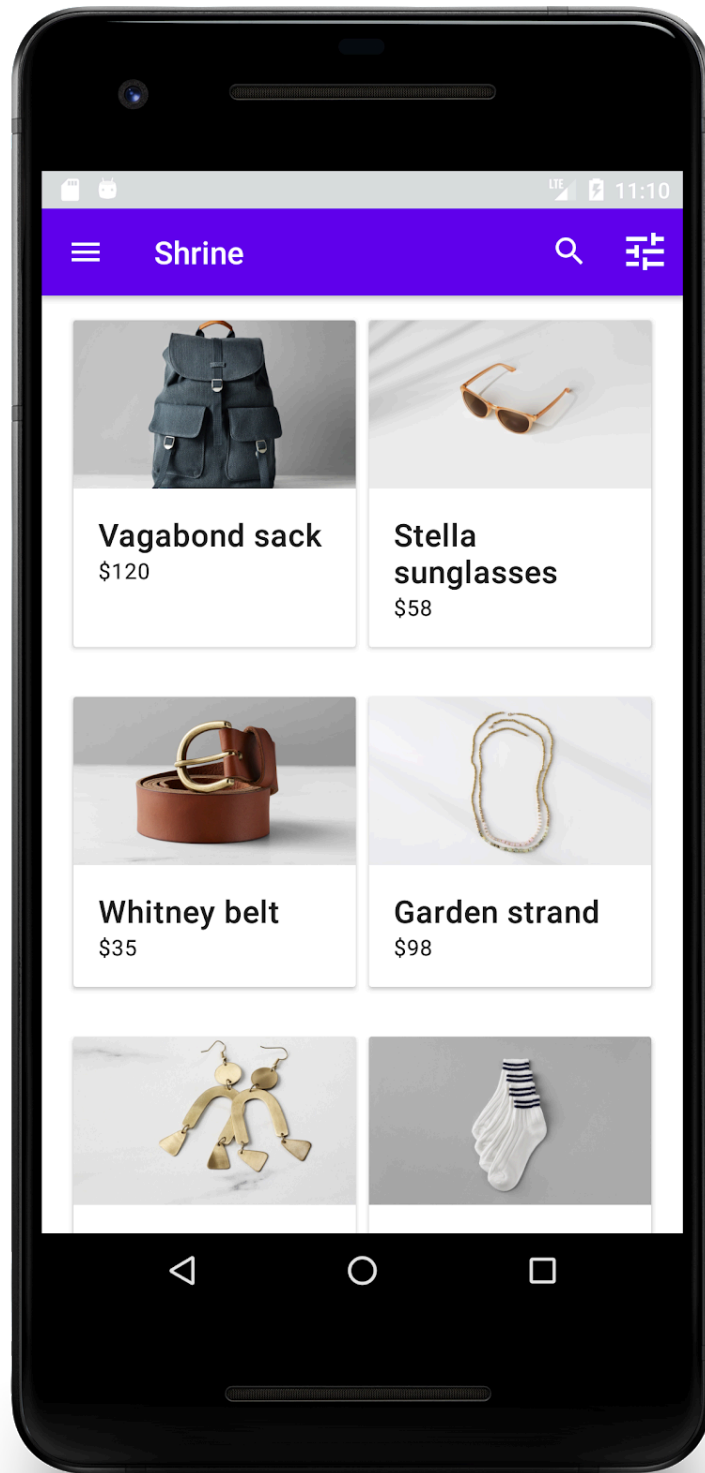
開發人員可透過 Material 元件 (MDC) 實作 Material Design。MDC 由 Google 的工程師和 UX 設計師團隊打造，提供數十種美觀實用的 UI 元件，適用於 Android、iOS、網頁和 Flutter。material.io/develop

在 MDC-101 程式碼研究室中，您使用了兩個 Material Design 元件 (MDC) 建構登入頁面：文字欄位和按鈕。現在，我們將新增導覽、結構和資料，擴充這個基礎。

建構項目

在本程式碼研究室中，您將為名為「**Shrine**」的應用程式建構主畫面。這款電子商務應用程式販售服飾和居家用品。其中包含：

- 頂端應用程式列
- 產品格狀清單



Vagabond sack
\$120



Stella sunglasses
\$58



Whitney belt
\$35



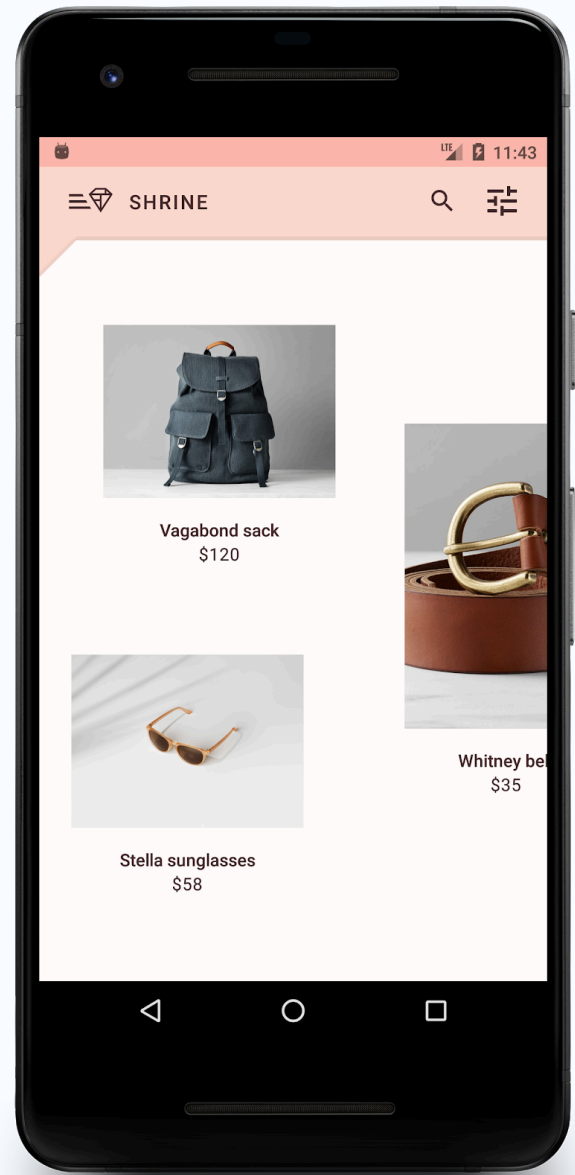
Garden strand
\$98



這是四個程式碼研究室的第二個，將引導您建構名為 Shrine 的產品應用程式。建議您依序完成所有程式碼研究室，逐步完成各項工作。相關程式碼研究室如下：

- [MDC-101：Material Design 元件 \(MDC\) 基本概念](#)
- [MDC-103：利用顏色、形狀、高度和類型建立 Material Design 主題](#)
- [MDC-104：Material Design 進階元件。](#)

完成 MDC-104 後，您將建構出如下所示的應用程式：



本程式碼研究室中的 MDC-Android 元件

- AppBarLayout
- MaterialCardView

在本程式碼研究室中，您將使用 MDC-Android 提供的預設元件。您將在 [MDC-103：利用顏色、形狀、高度和類型建立 Material Design 主題](#) 中，瞭解如何自訂這些屬性。

軟硬體需求

- 具備 Android 開發的基本知識

- **Android Studio** (如果沒有，請[在這裡](#)下載)
 - Android 模擬器或裝置 (可透過 Android Studio 取得)
 - 程式碼範例 (請參閱下一個步驟)
-

2. 設定開發環境

是否要從 MDC-101 繼續？

如果您已完成 MDC-101，程式碼應該已準備就緒，可供本程式碼研究室使用。您可以跳至步驟 3：新增頂端應用程式列。

從頭開始嗎？

下載程式碼研究室的入門應用程式

範例應用程式位於 `material-components-android-codelabs-102-starter/java` 目錄中。開始前，請務必 `cd` 到該目錄。

...或從 GitHub 複製

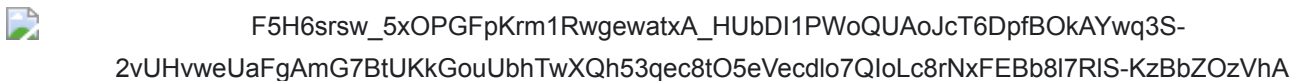
如要從 GitHub 複製本程式碼研究室，請執行下列指令：

```
git clone https://github.com/material-components/material-components-android-codelabs
cd material-components-android-codelabs/
git checkout 102-starter
```

如需更多說明，請參閱「[從 GitHub 複製存放區](#)」。

在 Android Studio 中載入範例程式碼

1. 設定精靈完成後，系統會顯示「Welcome to Android Studio」視窗，請按一下「Open an existing Android Studio project」。前往安裝程式碼範例的目錄，然後選取「java -> shrine」（或在電腦上搜尋「shrine」）開啟 Shrine 專案。
2. 稍待片刻，Android Studio 會建構及同步處理專案，Android Studio 視窗底部的活動指標會顯示進度。
3. 此時，Android Studio 可能會引發一些建構錯誤，因為您缺少 Android SDK 或建構工具，如下所示。按照 Android Studio 中的操作說明安裝/更新這些項目，然後同步處理專案。



F5H6srsW_5xOPGFpKrm1RwgewatxA_HUbDI1PWoQUAoJcT6DpfBOKAYwq3S-
2vUHvweUaFgAmG7BtUKkGouUbhTwXQh53qec8tO5eVecdIo7QIoLc8rNxFEbB8I7RIS-KzBbZOzVhA

新增專案依附元件

專案需要 **MDC Android** 支援程式庫的依附元件。您下載的程式碼範例應該已列出這項依附元件，但建議您執行下列步驟確認。

1. 前往 `app` 模組的 `build.gradle` 檔案，確認 `dependencies` 區塊包含 MDC Android 的依附元件：

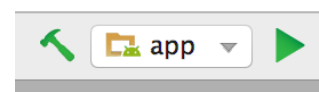
```
api 'com.google.android.material:material:1.1.0-alpha06'
```

2. (選用) 視需要編輯 `build.gradle` 檔案，加入下列依附元件並同步處理專案。

```
dependencies {
    api 'com.google.android.material:material:1.1.0-alpha06'
    implementation 'androidx.legacy:legacy-support-v4:1.0.0'
    implementation 'com.android.volley:volley:1.1.1'
    implementation 'com.google.code.gson:gson:2.8.5'
    implementation "org.jetbrains.kotlin:kotlin-stdlib-jdk7:1.3.21"
    testImplementation 'junit:junit:4.12'
    androidTestImplementation 'androidx.test:core:1.1.0'
    androidTestImplementation 'androidx.test.ext:junit:1.1.0'
    androidTestImplementation 'androidx.test:runner:1.2.0-alpha05'
    androidTestImplementation 'androidx.test.espresso:espresso-core:3.2.0-alpha05'
}
```

執行範例應用程式

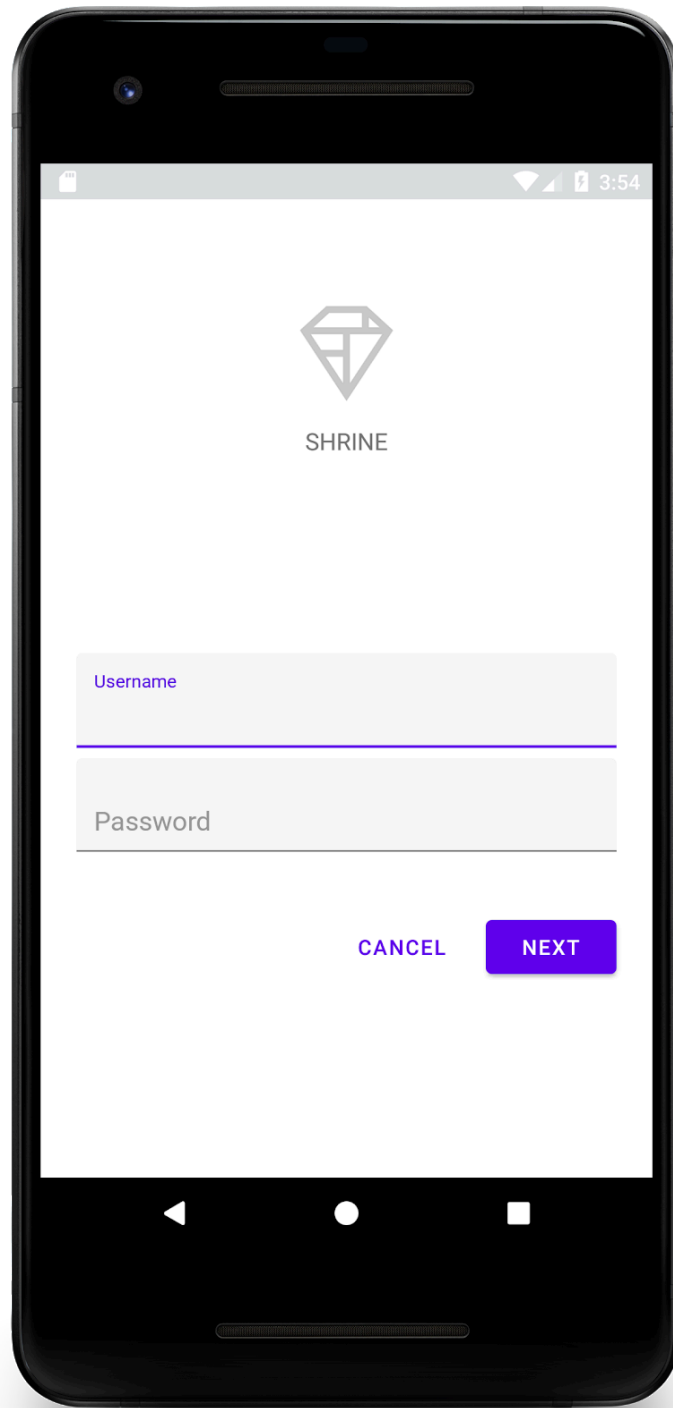
1. 確認「Run」/「Play」按鈕左側的建構設定為 `app`。
2. 按下綠色的「Run / Play」按鈕，即可建構及執行應用程式。
3. 在「Select Deployment Target」視窗中，如果可用裝置清單中已有 **Android** 裝置，請跳至步驟 8。如果沒有，請按一下「Create New Virtual Device」。
4. 在「Select Hardware」畫面中，選取手機裝置 (例如 **Pixel 2**)，然後按一下「Next」。
5. 在「系統映像檔」畫面中，選取最新的 **Android** 版本，最好是最高的 API 級別。如果尚未安裝，請按一下顯示的「下載」連結，然後完成下載。
6. 點選 [下一步]。
7. 在 **Android 虛擬裝置 (AVD)** 畫面中，保留預設設定並點選「Finish」。
8. 從部署目標對話方塊中選取 **Android** 裝置。
9. 按一下 [確定]。
10. Android Studio 會建構及部署應用程式，並在目標裝置上自動開啟。



如果無法新增依附元件或順利執行應用程式，請停止並排解開發人員環境的問題。

如果在建構時收到 `INSTALL_FAILED_INVALID_APK` 錯誤，請嘗試在 Android Studio 中依序選取「Build」(建構) -> 「Clean Project」(清除專案)。

太棒了，您應該會看到 MDC-101 程式碼研究室的 Shrine 登入頁面。



登入畫面現在看起來不錯，接下來請在應用程式中填入一些產品。

3. 新增頂端應用程式列

登入頁面關閉後，系統會顯示主畫面和「你成功了！」訊息。太好了！但現在使用者無法採取任何動作，也無法瞭解自己在應用程式中的位置。為解決這個問題，我們來新增導覽功能。

導覽是指可讓使用者在應用程式中移動的元件、互動、視覺提示和資訊架構。導覽有助於發掘內容和功能，方便使用者完成工作。

Material Design 提供導覽模式，確保高度可用性。頂端應用程式列是最顯眼的導覽元件之一。

頂端應用程式列在 iOS 中可能稱為「導覽列」，或簡稱為「應用程式列」或「標題」。

為了提供導覽功能，並讓使用者快速存取其他動作，請新增頂端應用程式列。

新增 AppBar 小工具

在 `shr_product_grid_fragment.xml` 中，刪除包含「你成功了！」的 `<LinearLayout>` 標記。 `TextView`，並替換為下列內容：

`shr_product_grid_fragment.xml`

```
<com.google.android.material.appbar.AppBarLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content">

    <androidx.appcompat.widget.Toolbar
        android:id="@+id/app_bar"
        style="@style/Widget.Shrine.Toolbar"
        android:layout_width="match_parent"
        android:layout_height="?attr/actionBarSize"
        app:title="@string/shr_app_name" />
</com.google.android.material.appbar.AppBarLayout>
```

`shr_product_grid_fragment.xml` 應如下所示：

`shr_product_grid_fragment.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".ProductGridFragment">

    <com.google.android.material.appbar.AppBarLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <androidx.appcompat.widget.Toolbar
            android:id="@+id/app_bar"
            style="@style/Widget.Shrine.Toolbar"
            android:layout_width="match_parent"
            android:layout_height="?attr/actionBarSize"
            app:title="@string/shr_app_name" />
        </com.google.android.material.appbar.AppBarLayout>

</FrameLayout>
```

許多應用程式列的標題旁都有按鈕。讓我們在其中新增選單圖示。

新增導覽圖示

在 `shr_product_grid_fragment.xml` 中，將下列項目新增至 `Toolbar` XML 元件 (您剛才新增至版面配置)：

shr_product_grid_fragment.xml

```
app:navigationIcon="@drawable/shr_menu"
```

`shr_product_grid_fragment.xml` 應如下所示：

shr_product_grid_fragment.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".ProductGridFragment">

    <com.google.android.material.appbar.AppBarLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <androidx.appcompat.widget.Toolbar
            android:id="@+id/app_bar"
            style="@style/Widget.Shrine.Toolbar"
            android:layout_width="match_parent"
            android:layout_height="?attr/actionBarSize"
            app:navigationIcon="@drawable/shr_menu"
            app:title="@string/shr_app_name" />
    </com.google.android.material.appbar.AppBarLayout>

</FrameLayout>
```

新增動作按鈕並設定頂端應用程式列的樣式

您也可以在應用程式列的尾端新增按鈕。在 Android 中，這些按鈕稱為「動作按鈕」。

「開始」和「結束」是表示方向的用語，以不限語言的方式指稱文字行的開頭和結尾。如果使用由左至右 (LTR) 的語言 (例如英文)，「start」表示「left」，「end」表示「right」。在阿拉伯文等 RTL (從右到左) 語言中，「start」是指「right」，「end」是指「left」。

如要進一步瞭解 UI 鏡像，請參閱「[雙向性](#)」的 Material Design 指南。

我們會設定頂端應用程式列的樣式，並以程式輔助方式將動作按鈕新增至選單。

首先，我們來建立設定工具列的方法。這個方法應使用工具列的 `id` 取得工具列的參照，並使用 `getActivity()` 取得活動的參照。如果活動不是空值，請使用 `setSupportActionBar` 將 `Toolbar` 設為要使用的 `ActionBar`：

ProductGridFragment.java

```
private void setUpToolbar(View view) {
    Toolbar toolbar = view.findViewById(R.id.app_bar);
    AppCompatActivity activity = (AppCompatActivity) getActivity();
    if (activity != null) {
        activity.supportActionBar(toolbar);
    }
}
```

接著，在剛才新增的 `setUpToolbar` 方法下方，覆寫 `onCreateOptionsMenu`，將 `shr_toolbar_menu.xml` 的內容加載至工具列：

ProductGridFragment.java

```
@Override
public void onCreateOptionsMenu(Menu menu, MenuInflater menuInflater) {
    menuInflater.inflate(R.menu.shr_toolbar_menu, menu);
    super.onCreateOptionsMenu(menu, menuInflater);
}
```

注意：如果 Android Studio 在編寫這段程式碼時發生「Cannot resolve symbol」（無法解析符號）錯誤，請按 **Alt + Enter** 鍵（或 Mac 上的 **Option + Return** 鍵）匯入缺少的符號。

重要事項：匯入 `Toolbar` 類別時，請務必匯入 `androidx.appcompat.widget.Toolbar`，而非 `android.widget.Toolbar`

現在，請在 `onCreateView()` 方法的內容中新增對 `setUpToolbar` 方法的呼叫，如下所示：

ProductGridFragment.java

```
@Override
public View onCreateView(
    @NonNull LayoutInflater inflater, ViewGroup container, Bundle savedInstanceState) {
    // Inflate the layout for this fragment with the ProductGrid theme
    View view = inflater.inflate(R.layout.shr_product_grid_fragment, container, false);

    // Set up the toolbar
    setUpToolbar(view);

    return view;
}
```

最後，將 `onCreate()` 方法新增至 `ProductGridFragment.java`。在方法主體中，將 `setHasOptionsMenu` 的參數設為 `true`。

方法應如下所示：

ProductGridFragment.java

```
@Override
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setHasOptionsMenu(true);
}
```

上述程式碼會將 XML 版面配置中的應用程式列設為這項活動的動作列。回呼 `onCreateOptionsMenu` 會告知活動要將哪些內容做為選單。在本例中，這會將 `R.menu.shr_toolbar_menu` 中的選單項目放入應用程式列。

選單檔案包含「搜尋」和「篩選」這兩個項目。

shr_toolbar_menu.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android"
      xmlns:app="http://schemas.android.com/apk/res-auto">
  <item
    android:id="@+id/search"
    android:icon="@drawable/shr_search"
    android:title="@string/shr_search_title"
    app:showAsAction="always" />
  <item
    android:id="@+id/filter"
    android:icon="@drawable/shr_filter"
    android:title="@string/shr_filter_title"
    app:showAsAction="always" />
</menu>
```

完成這些變更後，`ProductGridFragment.java` 檔案應如下所示：

ProductGridFragment.java

```

package com.google.codelabs.mdc.java.shrine;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.Menu;
import android.view.MenuInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Toolbar;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.fragment.app.Fragment;

public class ProductGridFragment extends Fragment {

    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setHasOptionsMenu(true);
    }

    @Override
    public View onCreateView(
        @NonNull LayoutInflater inflater, ViewGroup container, Bundle savedInstanceState)
    {
        // Inflate the layout for this fragment with the ProductGrid theme
        View view = inflater.inflate(R.layout.shr_product_grid_fragment, container, false);

        // Set up the toolbar
        setUpToolbar(view);

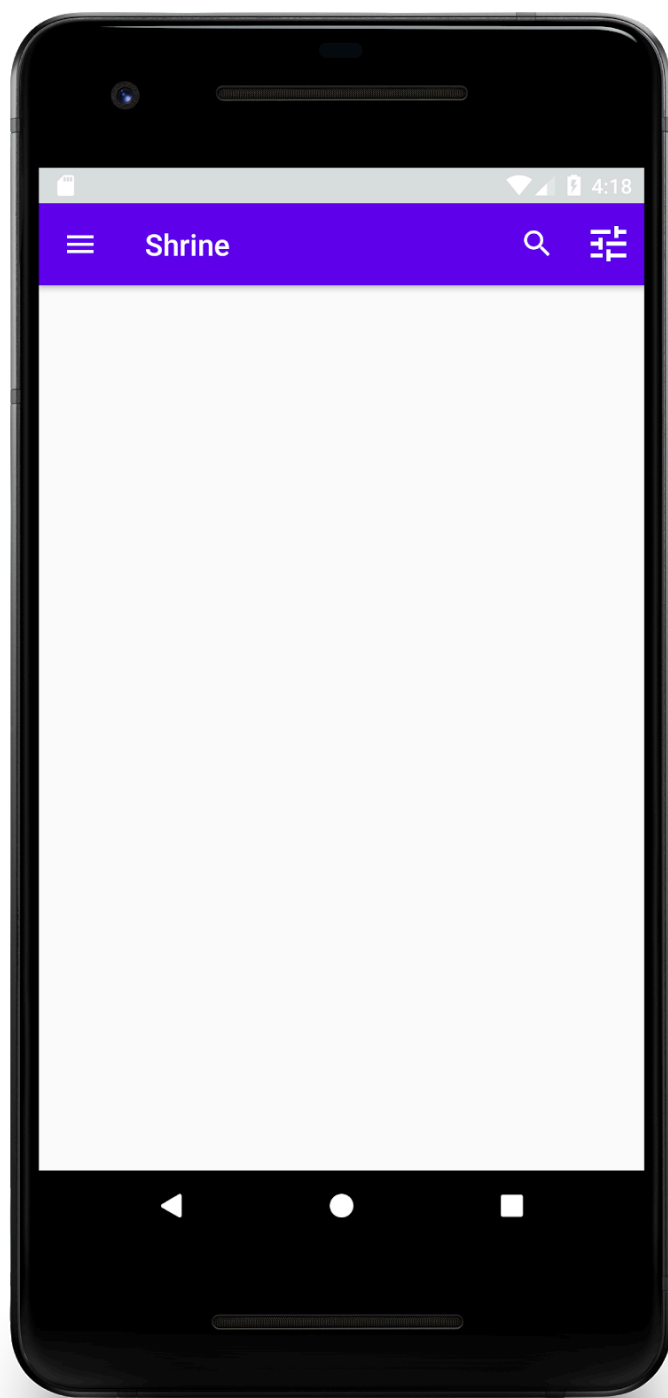
        return view;
    }

    private void setUpToolbar(View view) {
        Toolbar toolbar = view.findViewById(R.id.app_bar);
        AppCompatActivity activity = (AppCompatActivity) getActivity();
        if (activity != null) {
            activity.supportActionBar(toolbar);
        }
    }

    @Override
    public void onCreateOptionsMenu(Menu menu, MenuInflater menuInflater) {
        menuInflater.inflate(R.menu.shr_toolbar_menu, menu);
        super.onCreateOptionsMenu(menu, menuInflater);
    }
}

```

建構並執行。主畫面應如下所示：



現在工具列包含導覽圖示、標題，以及右側的兩個動作圖示。工具列也會使用細微的陰影顯示高度，表示工具列與內容位於不同圖層。

你會發現頂端的狀態列不相符。目前不必擔心這個問題，因為我們會在 MDC-103 中進行顏色主題設定時修正。

步驟 4 · 約 6 分鐘

4. 新增卡片

現在應用程式已具備一些結構，接下來請將內容放在資訊卡中，以便整理。

資訊卡

資訊卡是顯示單一主題內容和動作的元件。這類廣告可彈性呈現類似內容的集合。

新增資訊卡

首先，在頂端應用程式列下方新增一張資訊卡。資訊卡應包含圖片區域、標題和次要文字標籤。

在 `shr_product_grid_fragment.xml` 中，於 `AppBarLayout` 下方新增下列內容：

[shr_product_grid_fragment.xml](#)

```

<com.google.android.material.card.MaterialCardView
    android:layout_width="160dp"
    android:layout_height="180dp"
    android:layout_marginBottom="16dp"
    android:layout_marginLeft="16dp"
    android:layout_marginRight="16dp"
    android:layout_marginTop="70dp"
    app:cardBackgroundColor="?attr/colorPrimaryDark"
    app:cardCornerRadius="4dp">

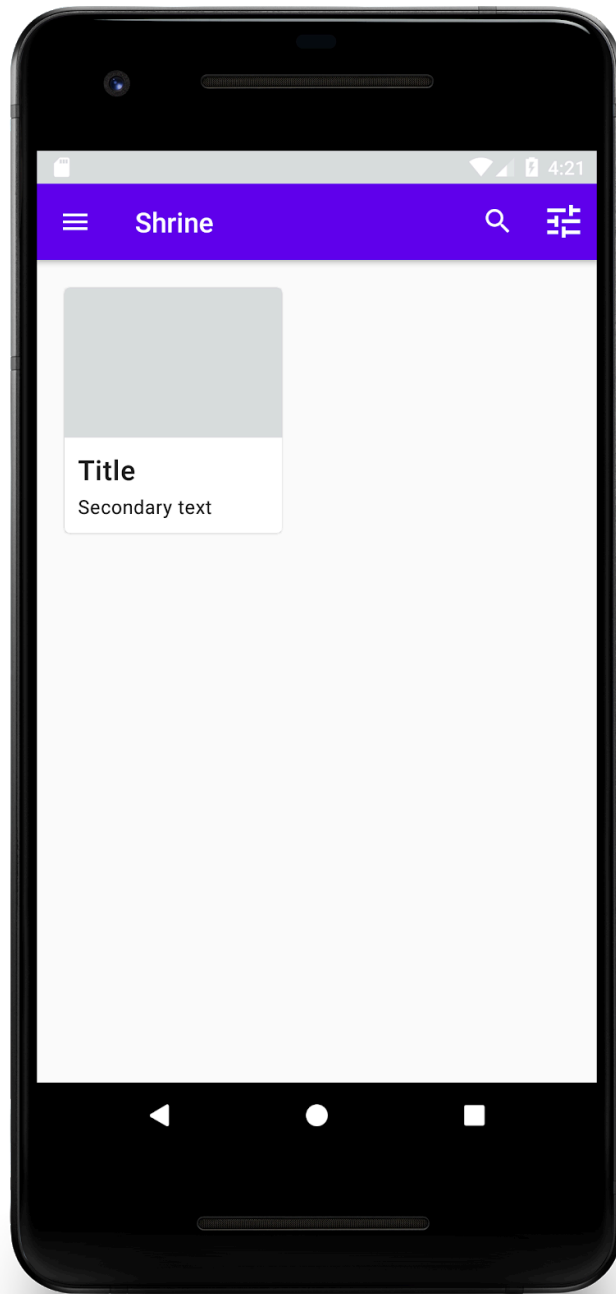
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_gravity="bottom"
        android:background="#FFFFFF"
        android:orientation="vertical"
        android:padding="8dp">

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:padding="2dp"
            android:text="@string/shr_product_title"
            android:textAppearance="?attr/textAppearanceHeadline6" />

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:padding="2dp"
            android:text="@string/shr_product_description"
            android:textAppearance="?attr/textAppearanceBody2" />
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

```

建構及執行：



從這個預覽畫面中，您可以看到卡片從螢幕左側邊緣內縮，且具有圓角和陰影 (表示卡片的高度)。整個區域稱為「容器」。除了容器本身，其中的所有元素都是選用。

您可以在容器中新增下列元素：標題文字、縮圖或個人資料相片、副標題文字、分隔線，甚至是按鈕和圖示。舉例來說，我們剛建立的資訊卡包含兩個 `TextView` (一個用於標題，另一個用於次要文字)，位於 `LinearLayout` 中，並與資訊卡底部對齊。

資訊卡通常會與其他資訊卡一起顯示在集合中。在本程式碼研究室的下一節中，我們會將這些圖片以格線集合的形式排列。

步驟 5 · 約 10 分鐘

5. 建立卡片格線

如果畫面中有多張資訊卡，系統會將這些資訊卡歸類為一或多個集合。格線中的資訊卡是共平面，也就是說，彼此的靜止高度相同 (除非資訊卡遭到選取或拖曳，但我們不會在本程式碼研究室中介紹這項功能)。

設定卡片網格

請查看我們提供的 `shr_product_card.xml` 檔案：

[shr_product_card.xml](#)

```

<?xml version="1.0" encoding="utf-8"?>
<com.google.android.material.card.MaterialCardView
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:cardBackgroundColor="@android:color/white"
    app:cardElevation="2dp"
    app:cardPreventCornerOverlap="true">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <com.android.volley.toolbox.NetworkImageView
            android:id="@+id/product_image"
            android:layout_width="match_parent"
            android:layout_height="@dimen/shr_product_card_image_height"
            android:background="?attr/colorPrimaryDark"
            android:scaleType="centerCrop" />

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="16dp">

            <TextView
                android:id="@+id/product_title"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:text="@string/shr_product_title"
                android:textAppearance="?attr/textAppearanceHeadline6" />

            <TextView
                android:id="@+id/product_price"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:text="@string/shr_product_description"
                android:textAppearance="?attr/textAppearanceBody2" />

        </LinearLayout>
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

```

這個資訊卡版面配置包含一張圖片資訊卡 (這裡的 `NetworkImageView` 可讓我們從網址加入圖片)，以及兩個 `TextViews`。

接著，請查看我們為你提供的 `ProductCardRecyclerViewAdapter`。與 `ProductGridFragment` 位於同一個套件中。

ProductCardRecyclerViewAdapter.java

```

package com.google.codelabs.mdc.java.shrine;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.google.codelabs.mdc.java.shrine.network.ImageRequester;
import com.google.codelabs.mdc.java.shrine.network.ProductEntry;

import java.util.List;

/**
 * Adapter used to show a simple grid of products.
 */
public class ProductCardRecyclerViewAdapter extends
    RecyclerView.Adapter<ProductCardViewHolder> {

    private List<ProductEntry> productList;
    private ImageRequester imageRequester;

    ProductCardRecyclerViewAdapter(List<ProductEntry> productList) {
        this.productList = productList;
        imageRequester = ImageRequester.getInstance();
    }

    @NonNull
    @Override
    public ProductCardViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
        View layoutView =
            LayoutInflater.from(parent.getContext()).inflate(R.layout.shr_product_card, parent, false);
        return new ProductCardViewHolder(layoutView);
    }

    @Override
    public void onBindViewHolder(@NonNull ProductCardViewHolder holder, int position) {
        // TODO: Put ViewHolder binding code here in MDC-102
    }

    @Override
    public int getItemCount() {
        return productList.size();
    }
}

```

上述的介面卡類別會管理格線的內容。為了決定每個檢視區塊應如何處理指定內容，我們很快就會編寫 `onBindViewHolder()` 的程式碼。

在同一個套件中，您也可以查看 `ProductCardViewHolder`。這個類別會儲存影響資訊卡版面配置的檢視區塊，方便我們稍後修改。

ProductCardViewHolder.java

```
package com.google.codelabs.mdc.java.shrine;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;
import android.view.View;

public class ProductCardViewHolder extends RecyclerView.ViewHolder {

    public ProductCardViewHolder(@NonNull View itemView) {
        super(itemView);
        // TODO: Find and store views from itemView
    }
}
```

如要設定格線，首先要從 `shr_product_grid_fragment.xml` 移除 `shr_product_grid_fragment.xml` 中的預留位置。MaterialCardView 接著，您應新增代表卡片格線的元件。在本例中，請在 AppBarLayout XML 元件下方的 `shr_product_grid_fragment.xml` 中新增 RecyclerView 元件：

shr_product_grid_fragment.xml

```
<androidx.core.widget.NestedScrollView
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_marginTop="56dp"
    android:background="@color/productGridBackgroundColor"
    android:paddingStart="@dimen/shr_product_grid_spacing"
    android:paddingEnd="@dimen/shr_product_grid_spacing"
    app:layout_behavior="@string/appbar_scrolling_view_behavior">

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recycler_view"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />

</androidx.core.widget.NestedScrollView>
```

`shr_product_grid_fragment.xml` 應如下所示：

shr_product_grid_fragment.xml

```

<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".ProductGridFragment">

    <com.google.android.material.appbar.AppBarLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <androidx.appcompat.widget.Toolbar
            android:id="@+id/app_bar"
            style="@style/Widget.Shrine.Toolbar"
            android:layout_width="match_parent"
            android:layout_height="?attr/actionBarSize"
            app:navigationIcon="@drawable/shr_menu"
            app:title="@string/shr_app_name" />
    </com.google.android.material.appbar.AppBarLayout>

    <androidx.core.widget.NestedScrollView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_marginTop="56dp"
        android:background="@color/productGridBackgroundColor"
        android:paddingStart="@dimen/shr_product_grid_spacing"
        android:paddingEnd="@dimen/shr_product_grid_spacing"
        app:layout_behavior="@string/appbar_scrolling_view_behavior">

        <androidx.recyclerview.widget.RecyclerView
            android:id="@+id/recycler_view"
            android:layout_width="match_parent"
            android:layout_height="match_parent" />

    </androidx.core.widget.NestedScrollView>

</FrameLayout>

```

最後，在 `onCreateView()` 中，於呼叫 `setUpToolbar(view)` 後和 `return` 陳述式前，將 `RecyclerView` 初始化程式碼新增至 `ProductGridFragment.java`：

ProductGridFragment.java


```

@Override
public View onCreateView(
    @NonNull LayoutInflater inflater, ViewGroup container, Bundle savedInstanceState) {
    ...
    setUpToolbar(view);

    // Set up the RecyclerView
    RecyclerView recyclerView = view.findViewById(R.id.recycler_view);
    recyclerView.setHasFixedSize(true);
    recyclerView.setLayoutManager(new GridLayoutManager(getContext(), 2,
GridLayoutManager.VERTICAL, false));
    ProductCardRecyclerViewAdapter adapter = new ProductCardRecyclerViewAdapter(
        ProductEntry.initProductEntryList(getResources()));
    recyclerView.setAdapter(adapter);
    int largePadding = getResources().getDimensionPixelSize(R.dimen.shr_product_grid_spacing);
    int smallPadding =
getResources().getDimensionPixelSize(R.dimen.shr_product_grid_spacing_small);
    recyclerView.addItemDecoration(new ProductGridItemDecoration(largePadding, smallPadding));

    return view;
}

```

上述程式碼片段包含設定 `RecyclerView` 的必要初始化步驟。包括設定 `RecyclerView` 的版面配置管理工具，以及初始化及設定 `RecyclerView` 的轉接器。

您的 `ProductGridFragment.java` 檔案現在應如下所示：

ProductGridFragment.java

```

package com.google.codelabs.mdc.java.shrine;

import android.os.Bundle;
import androidx.recyclerview.widget.RecyclerView;
import android.view.LayoutInflater;
import android.view.Menu;
import android.view.MenuInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Toolbar;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.fragment.app.Fragment;
import androidx.recyclerview.widget.GridLayoutManager;

import com.google.codelabs.mdc.java.shrine.network.ProductEntry;

public class ProductGridFragment extends Fragment {

    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setHasOptionsMenu(true);
    }

    @Override
    public View onCreateView(
        @NonNull LayoutInflater inflater, ViewGroup container, Bundle savedInstanceState)
    {
        // Inflate the layout for this fragment with the ProductGrid theme
        View view = inflater.inflate(R.layout.shr_product_grid_fragment, container, false);

        // Set up the toolbar
        setUpToolbar(view);

        // Set up the RecyclerView
        RecyclerView recyclerView = view.findViewById(R.id.recycler_view);
        recyclerView.setHasFixedSize(true);
        recyclerView.setLayoutManager(new GridLayoutManager(getContext(), 2,
GridLayoutManager.VERTICAL, false));
        ProductCardRecyclerViewAdapter adapter = new ProductCardRecyclerViewAdapter(
            ProductEntry.initProductEntryList(getResources()));
        recyclerView.setAdapter(adapter);
        int largePadding =
getResources().getDimensionPixelSize(R.dimen.shr_product_grid_spacing);
        int smallPadding =
getResources().getDimensionPixelSize(R.dimen.shr_product_grid_spacing_small);
        recyclerView.addItemDecoration(new ProductGridItemDecoration(largePadding,
smallPadding));

        return view;
    }

```

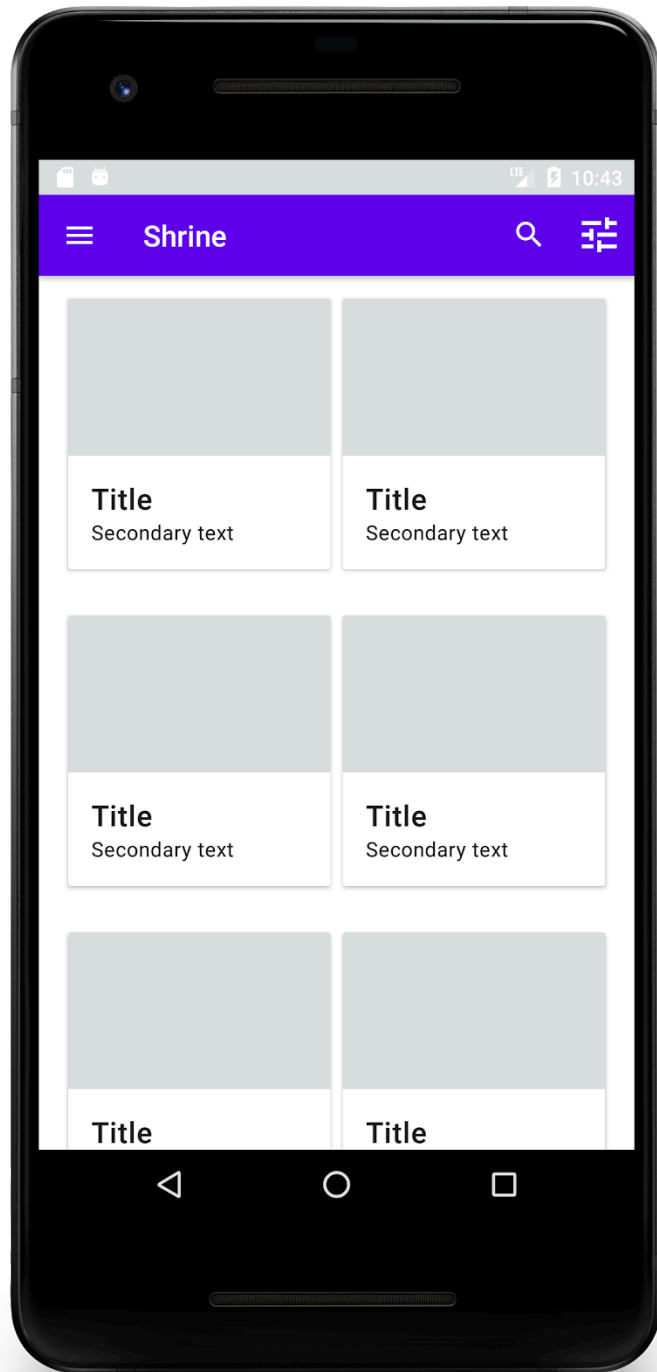
```
}

private void setUpToolbar(View view) {
    Toolbar toolbar = view.findViewById(R.id.app_bar);
    AppCompatActivity activity = (AppCompatActivity) getActivity();
    if (activity != null) {
        activity.supportActionBar(toolbar);
    }
}

@Override
public void onCreateOptionsMenu(Menu menu, MenuInflater menuInflater) {
    menuInflater.inflate(R.menu.shr_toolbar_menu, menu);
    super.onCreateOptionsMenu(menu, menuInflater);
}

}
```

建構並執行。



現在卡片已顯示在畫面中！目前還不會顯示任何內容，因此請新增一些產品資料。

新增圖片和文字

為每張資訊卡新增圖片、產品名稱和價格。我們的 `ViewHolder` 抽象化會保留每張資訊卡的檢視畫面。在 `ViewHolder` 中，新增下列三個檢視畫面：

`ProductCardViewHolder.java`

```

package com.google.codelabs.mdc.java.shrine;

import androidx.recyclerview.widget.RecyclerView;
import android.view.View;
import android.widget.TextView;
import androidx.annotation.NonNull;

import com.android.volley.toolbox.NetworkImageView;

public class ProductCardViewHolder extends RecyclerView.ViewHolder {

    public NetworkImageView productImage;
    public TextView productTitle;
    public TextView productPrice;

    public ProductCardViewHolder(@NonNull View itemView) {
        super(itemView);
        productImage = itemView.findViewById(R.id.product_image);
        productTitle = itemView.findViewById(R.id.product_title);
        productPrice = itemView.findViewById(R.id.product_price);
    }
}

```

在 `RecyclerView` 的介面卡中，更新 `onBindViewHolder()` 方法，在每個檢視區塊中設定資訊：`ViewHolder`,

ProductCardRecyclerViewAdapter.java

```

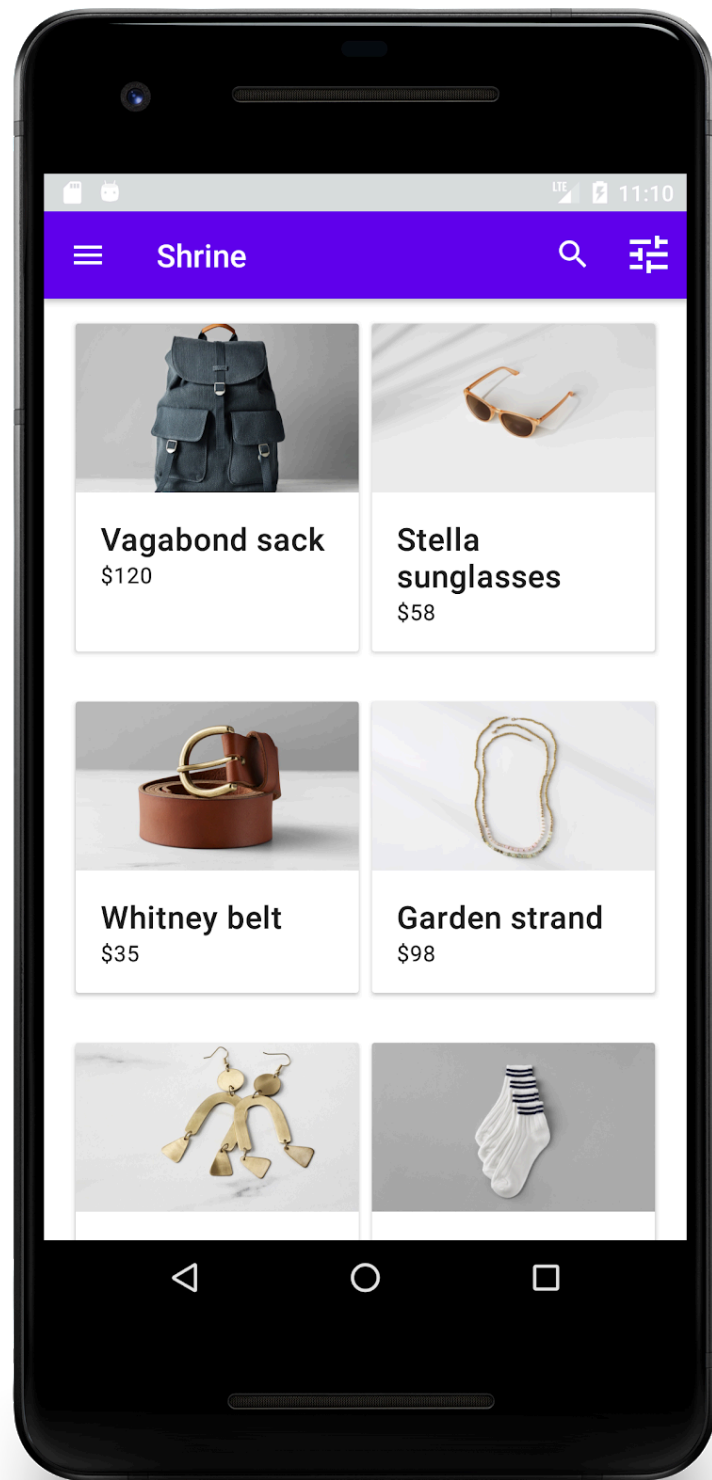
@Override
public void onBindViewHolder(@NonNull ProductCardViewHolder holder, int position) {
    if (productList != null && position < productList.size()) {
        ProductEntry product = productList.get(position);
        holder.productTitle.setText(product.title);
        holder.productPrice.setText(product.price);
        imageRequester.setImageFromUrl(holder.productImage, product.url);
    }
}

```

上述程式碼會使用 `ViewHolder`，告知 `RecyclerView` 的轉接程式如何處理每張資訊卡。

這裡會在每個 `ViewHolder` 的 `TextView` 上設定文字資料，並呼叫 `ImageRequester` 從網址取得圖片。`ImageRequester` 是我們為方便您而提供的類別，並使用 `volley` 程式庫 (這是本程式碼研究室範圍外的主題，但歡迎自行探索程式碼)。

建構及執行：



現在應用程式中會顯示我們的產品！

6. 重點回顧

我們的應用程式有基本流程，可將使用者從登入畫面帶往主畫面，方便他們查看產品。我們只用了幾行程式碼，就新增了頂端應用程式列，其中包含標題和三個按鈕，以及用來呈現應用程式內容的資訊卡格線。現在的主畫面簡潔實用，基本架構和可執行的內容一目瞭然。

如要取得已完成的 MDC 102 應用程式，請下載[已完成的程式碼研究室](#)，或使用 Git 簽出 102-complete：`git checkout 102-complete`。

您可以根據該分支版本測試應用程式。

後續步驟

我們已使用 MDC-Android 程式庫中的四個核心 Material Design 元件，包括頂端應用程式列、資訊卡、文字欄位和按鈕！您可以在 MDC-Android 目錄中探索更多元件，請參閱「[MDC Android 中的元件](#)」。

雖然功能齊全，但我們的應用程式尚未展現任何特定品牌。在 [MDC-103：利用顏色、形狀、高度和類型建立 Material Design 主題](#) 中，我們會自訂這些元件的樣式，展現充滿活力、現代感的品牌。
